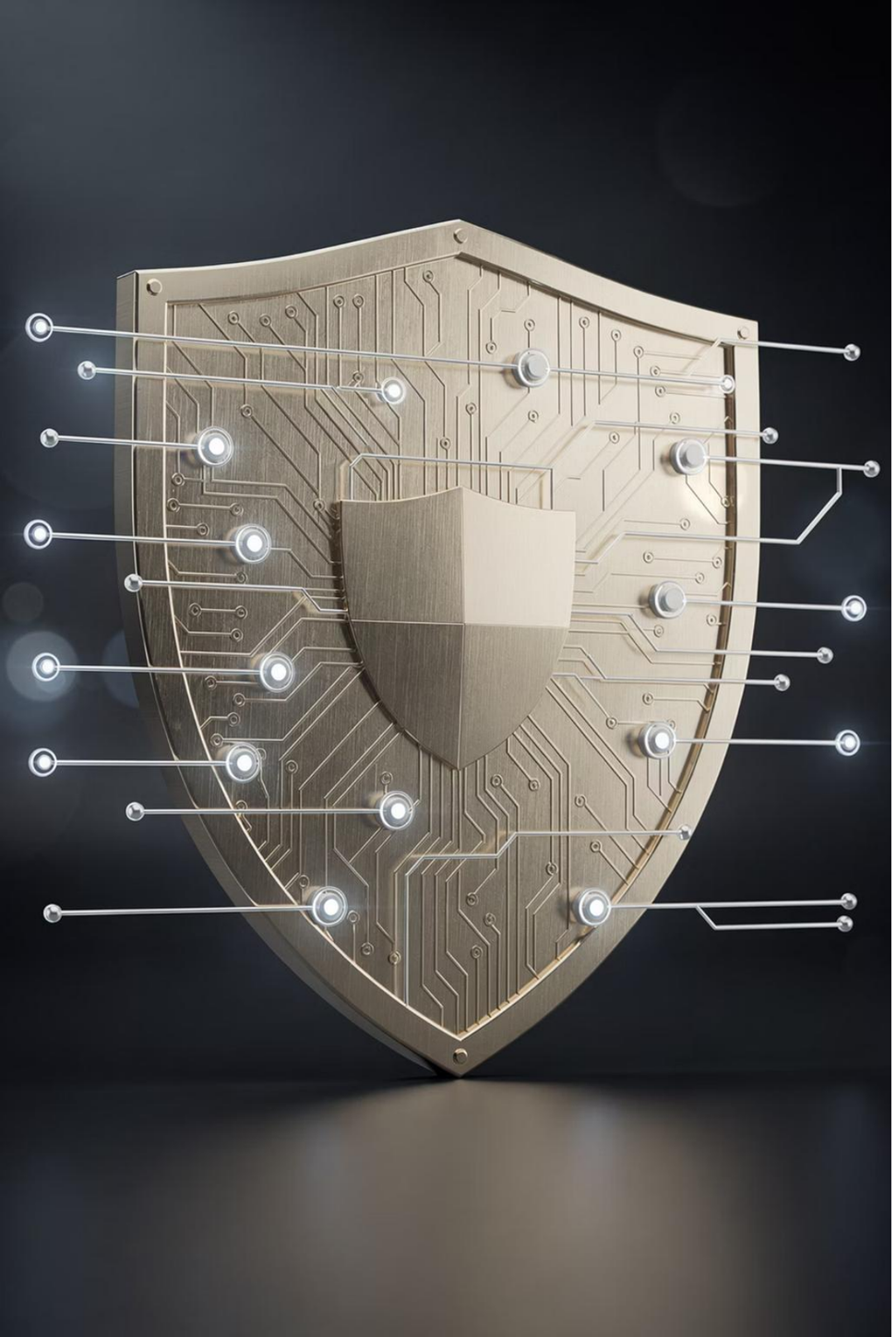


API Security Testing & DAST

Dynamic Application Security Testing (DAST) is used to test running applications and APIs to detect security vulnerabilities from an external perspective.

Introduction

APIs at the Core of Modern Applications

Modern applications rely heavily on APIs to exchange data between web services, mobile applications, cloud systems, and third-party platforms.

A Primary Target for Cyberattacks

Since APIs often expose business logic and sensitive data, they have become a primary target for cyberattacks.

The Need for Strong Security Testing

Organisations therefore need strong security testing strategies to identify vulnerabilities before systems are deployed to production environments.

Introduction

One important method used for testing live applications is **Dynamic Application Security Testing (DAST)**.

How DAST Works

DAST analyzes a running application from an external perspective, similar to how attackers interact with a system.

Instead of examining the source code, DAST tools send requests to API endpoints and analyze responses to detect vulnerabilities.

OWASP Recommendation

Security organisations such as **OWASP recommend dynamic security testing to detect issues defined in the OWASP API Security Top 10.**

Dynamic Application Security Testing (DAST)

Dynamic Application Security Testing is a security testing methodology that evaluates applications while they are running.

Black Box Approach

The testing approach treats the application as a **black box** and interacts with it through **HTTP requests and responses.**

Runtime Evaluation

Applications are evaluated while they are actively running, reflecting real-world conditions and live system behaviour.

Key Characteristics of DAST

1. Black-box testing method
2. Tests applications in runtime environment
3. Does not require access to source code
4. Simulates real attack scenarios
5. Detects vulnerabilities in deployed systems

Main Goal of DAST

The main objective is to identify vulnerabilities such as:

- SQL Injection
- Authentication Weaknesses
- Misconfigured Servers
- Input Validation Issues
- Authorisation Problems

Importance of DAST for API Security

APIs expose endpoints that accept external requests.

Attackers can exploit vulnerabilities in these endpoints if they are not properly tested.

Common API Security Risks Detected by DAST

- Injection attacks
- Broken authentication
- Broken object level authorisation
- Sensitive data exposure
- Security misconfiguration
- Rate-limiting weaknesses

Example Scenario

A DAST scanner automatically sends such payloads and checks whether the system is vulnerable.

API Endpoint

```
GET /api/products?name=laptop
```

Possible Malicious Request

```
GET /api/products?name=laptop' OR '1'='1
```

A DAST scanner automatically sends such payloads and checks whether the system is vulnerable to SQL injection and other attack patterns.

How DAST Works for APIs

DAST tools perform automated tests on API endpoints by simulating attack patterns.

DAST Testing Process

API Discovery

Identify available endpoints

Request Generation

Create various request combinations

Attack Simulation

Send malicious payloads

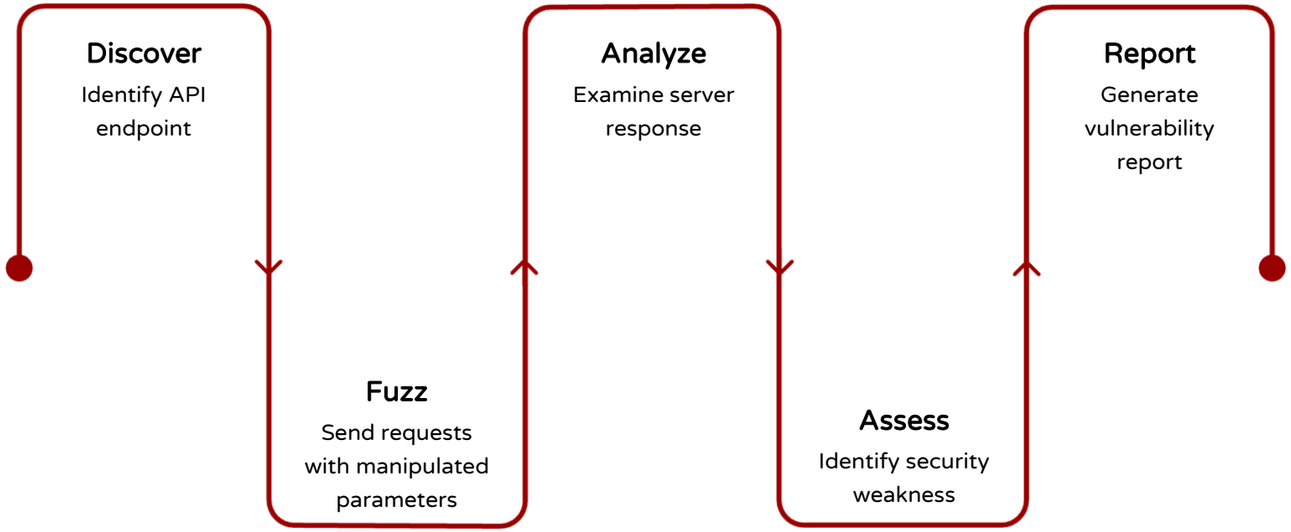
Response Analysis

Examine responses for vulnerabilities

Reporting

Generate vulnerability reports

Example Testing Workflow



A structured approach to dynamic application security testing ensures thorough coverage of API vulnerabilities from discovery through reporting.

Common DAST Tools for API Security

Several automated tools are available for performing dynamic security testing.

Popular DAST Tools

Tool	Type	Key Features
OWASP ZAP	Open-source	Automated vulnerability scanning
Burp Suite	Commercial	Advanced penetration testing
Nikto	Open-source	Web server misconfiguration detection
Acunetix	Enterprise	Automated vulnerability detection

Typical Capabilities of DAST Tools

- Automated Vulnerability Scanning
- API Endpoint Testing
- Injection Attack Simulation
- Security Misconfiguration Detection
- Report Generation

Advantages of DAST

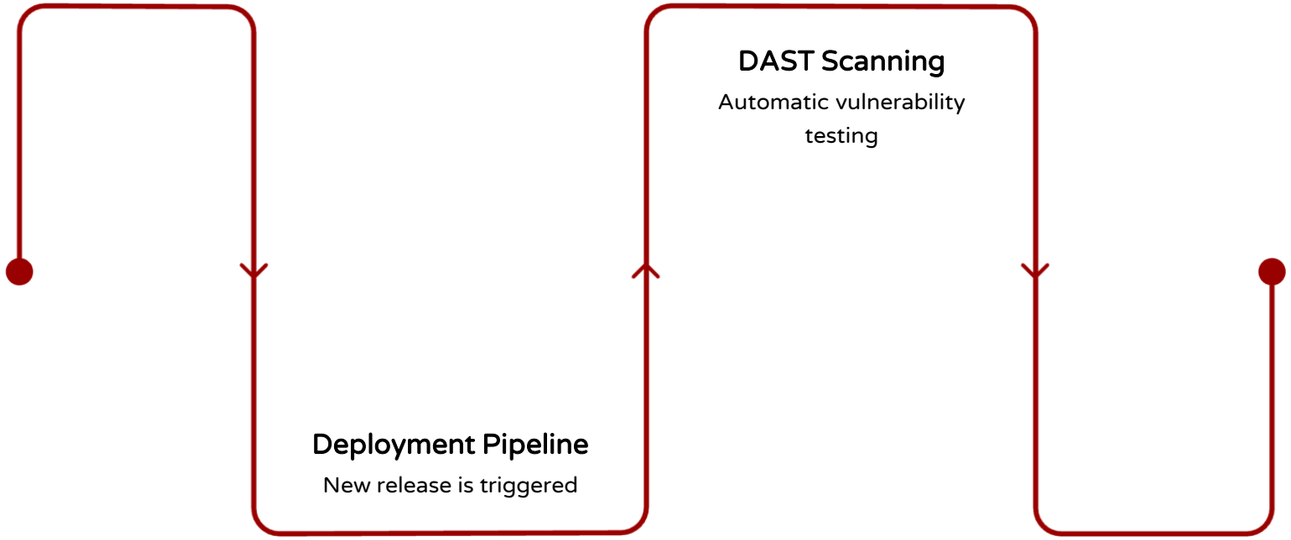
Dynamic security testing provides several benefits for developers and security teams.

Key Advantages

- → *Detects vulnerabilities in running applications*
- → *Simulates real attacker behaviour*
- → *Does not require source code access*
- → *Supports automated security testing*
- → *Identifies configuration issues*

Example Use Case

A software company integrates **DAST scanning into its deployment pipeline** so that each new release is automatically tested for security vulnerabilities.



Embedding DAST into the deployment pipeline ensures security is validated continuously with every release cycle.

Limitations of DAST

Although DAST is highly useful, it has certain limitations.

Common Limitations

- Limited visibility into source code
- Cannot identify exact code location of vulnerability
- Requires running application environment
- May produce false positives
- Some vulnerabilities may be missed

Best Practice for Security Testing

DAST should be combined with other security testing approaches such as:

Static Application Security Testing (SAST)

Analyses source code for vulnerabilities without executing the application.

Manual Penetration Testing

Human-led testing that uncovers complex vulnerabilities automated tools may miss.

Security Code Reviews

Systematic examination of source code to identify security flaws early in development.

Internal Testing Standards for API Security

Organisations often create internal security policies to ensure all applications meet security standards before deployment.

Common Internal Testing Standards

01

Mandatory security testing before release

02

Automated vulnerability scanning

03

Manual penetration testing

04

Secure coding guidelines

05

Continuous monitoring and logging

Example Internal Security Policy

Before deploying an API, the organisation requires:

- DAST Scan Completed
- No Critical Vulnerabilities Detected
- Authentication Mechanisms Verified
- Rate Limiting Enabled
- Security Headers Implemented

Benefits of Internal Standards

↑

Improve software security quality

Consistent standards raise the baseline security of every application released.

⚡

Detect vulnerabilities early

Catching issues before production dramatically reduces remediation cost and effort.

↓

Reduce production security incidents

Fewer vulnerabilities reaching live environments means fewer breaches and outages.

✓

Standardise security practices

Uniform processes ensure every team follows the same rigorous security approach.

Third-Party Security Requirements

In addition to internal policies, organizations must comply with external security standards defined by regulators, partners, or industry organizations.

Common Third-Party Security Requirements

- Regulatory compliance standards
- Industry security certifications
- Partner security audits
- Cloud security guidelines

Example Regulatory Authority

Example Regulatory Authority

Payment Card Industry Security Standards Council defines security requirements for payment systems.

Typical Third-Party

- Regular vulnerability scanning
- Annual penetration testing
- Secure authentication mechanisms
- Data protection controls
- Incident reporting procedures

Comparison: Internal Standards vs Third-Party Requirements

Factor	Internal Testing Standards	Third-Party Requirements
Purpose	Improve internal security	Meet regulatory compliance
Authority	Defined by organization	Defined by external entity
Flexibility	Customizable	Strict compliance required
Scope	Organisation-specific	Industry-wide
Example	Company security policy	Payment security compliance